TRABALHO PRÁTICO • 2026.2

DIVISÃO E CONQUISTA

Implementação, testes e análise experimental

Instituição: Universidade de Fortaleza (UNIFOR)

Disciplina: Complexidade e Análise de Algoritmos

Integrante 1: Yuri Freitas Andrade Maia — 2510459

Integrante 2: [PREENCHER NOME E MATRÍCULA]

Data: 27 de outubro de 2026 | **Entrega:** 28 de outubro de 2026

Resumo

Comparam-se quatro implementações em dois problemas: subarranjo de soma máxima e par de pontos mais próximo. Cada problema possui uma solução quadrática por força bruta e uma solução recursiva de divisão e conquista com ordem Θ(n log n). A validade foi examinada por casos de borda e testes diferenciais. Os tempos são medições efetivas em sete dimensões de entrada, com cinco execuções por combinação, registradas integralmente em CSV. O relatório apresenta as recorrências, a justificativa dos sete vizinhos, a escolha experimental do caso-base e as limitações do ambiente.

Escopo: exclusivamente Etapas 1, 2 e 3 do enunciado; a questão desafio foi excluída, conforme solicitado.

1. Subarranjo de soma máxima

A entrada é um inteiro n seguido de n valores inteiros, positivos ou negativos. A saída é uma tripla (soma máxima, início, fim), em índices baseados em zero e ambos inclusivos. O subarranjo deve ser não vazio. Para desempatar somas iguais, adota-se o menor índice inicial e, persistindo o empate, o menor índice final.

1.1. Solução por força bruta

O algoritmo fixa cada posição inicial e estende incrementalmente o limite final, atualizando a soma sem refazer a varredura. Examina todos os n(n + 1)/2 intervalos. Sua recorrência equivalente de custo é **$T(n) = T(n - 1) + \Theta(n)$**, com T(1) = Θ(1); portanto, **$T(n) = \Theta(n^2)$**. O espaço auxiliar é Θ(1).

1.2. Solução por divisão e conquista

Divide-se o vetor em duas metades, resolve-se a melhor soma em cada uma recursivamente e avalia-se o melhor intervalo que atravessa a posição central. A combinação faz uma passagem pela metade esquerda para obter o melhor sufixo e outra pela metade direita para obter o melhor prefixo. A melhor das três respostas resolve o problema completo.

A recorrência é **$T(n) = 2T(n/2) + \Theta(n)$**, com T(1) = Θ(1). Pelo caso 2 do Teorema Mestre, $n^{\log_2 2} = n$ e f(n) = Θ(n), logo **$T(n) = \Theta(n \log n)$**. Profundidade de pilha: Θ(log n), com espaço adicional Θ(log n), além da entrada.

1.3. Argumento de correção

Qualquer intervalo ótimo está integralmente à esquerda, integralmente à direita ou cruza a divisão. Por hipótese indutiva, os dois primeiros casos são calculados corretamente. No terceiro, toda solução que cruza o meio é a soma de um sufixo da esquerda e de um prefixo da direita, ambos maximizados pelas varreduras. Assim, escolher o melhor dos três candidatos é suficiente.

1.4. Testes críticos

Situação	Comportamento esperado
Todos negativos	Selecionar o maior elemento, sem aceitar intervalo vazio.
n = 1	Retornar o único índice e o próprio valor.
Empates	Aplicar início menor; em seguida fim menor.
Intervalo cruzado	Combinar prefixo e sufixo corretos.

2. Par de pontos mais próximo

Cada estação é um par (x, y). A saída informa dois índices distintos da entrada, sua distância euclidiana e seu quadrado. As distâncias são comparadas ao quadrado, evitando raízes desnecessárias e preservando precisão para coordenadas inteiras.

2.1. Solução por força bruta

Cada ponto é comparado com todos os seguintes: há $n(n-1)/2$ pares. Uma recorrência para o custo das comparações é $\mathbf{T(n) = T(n-1) + \Theta(n)}$ (base $n = 2$), que se resolve em $\mathbf{\Theta(n^2)}$; espaço auxiliar $\Theta(1)$.

2.2. Divisão e conquista em O(n log n)

Os pontos são ordenados inicialmente por x e por y. A lista por x é dividida na mediana; para cada chamada recursiva, a sequência ordenada por y é particionada em uma única passagem. Obtém-se a menor distância δ das duas metades, constrói-se uma faixa $|x - x_{mediana}| \leq \delta$ e percorrem-se os pontos ordenados por y. Cada ponto é comparado apenas aos sete seguintes, descartando candidatos cuja diferença vertical já exceda a distância corrente.

A combinação custa $\Theta(n)$, pois mantém as listas ordenadas por y, realiza particionamentos lineares e faz no máximo 7 comparações por ponto. Portanto, $\mathbf{T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)}$. As ordenações iniciais também têm custo $O(n \log n)$; o espaço auxiliar total é $O(n)$.

2.3. Por que bastam sete vizinhos?

Para um ponto p na faixa, só interessam sucessores em y separados por no máximo δ. Esses pontos cabem em um retângulo de largura 2δ e altura δ imediatamente acima de p. Corte-o pela linha mediana em duas regiões de largura δ; subdivida cada uma em quatro quadrados de lado δ/2. Existem oito quadrados. Como a diagonal de cada quadrado é $\delta/\sqrt{2}$, inferior a δ, não cabem dois pontos da mesma metade num mesmo quadrado: isso contradiz a definição de δ como menor distância interna. Assim, incluindo p, há no máximo oito pontos nesse retângulo, isto é, no máximo sete possíveis sucessores a verificar.

2.4. Duplicatas e caso-base

Pontos coincidentes são detectados na lista inicialmente ordenada por x e produzem distância zero (ótima global), evitando tratar $\delta = 0$ como uma faixa degenerada. Para subproblemas de até um limiar constante, a busca exaustiva reduz overhead recursivo sem alterar a ordem assintótica. O limiar foi selecionado por calibração real descrita na Seção 4.

3. Metodologia de avaliação

3.1. Instrumentação e repetibilidade

Foram executadas **5 repetições independentes de cronometragem por tamanho e por método**. O cronômetro **time.perf_counter_ns()** mede apenas a execução do algoritmo: geração de dados, validação prévia e aquecimento são externos ao intervalo cronometrado. São armazenados tempos em nanossegundos e calculados média, desvio amostral, mínimo e máximo em milissegundos.

As dimensões foram n = 100, 200, 400, 800, 1.600, 3.200 e 6.400. No enunciado aparece a palavra “dígitos” para a Etapa 2; ela foi interpretada como **quantidade de pontos**, coerente com o problema. Para cada valor de n, ambas as estratégias recebem exatamente o mesmo conjunto de dados, evitando diferenças de dificuldade entre execuções.

3.2. Perfil das entradas

Problema	Geração determinística
Subarranjo	n inteiros uniformes no intervalo [−100, 100]; semente 120000 + n.
Pontos	n pares de inteiros sem duplicatas, x e y entre -10^6 e 10^6; semente 220000 + n; ordem embaralhada.

3.3. Máquina utilizada nas medições

Item	Ambiente observado
Processador (identificação exposta)	AMD EPYC 9V74 80-Core Processor
CPU disponível	5 CPUs lógicas visíveis; quota cgroup: 400000 100000
Memória limitada por cgroup	4 GiB
Sistema operacional	Linux-6.18.55-x86_64-with-glibc2.41
Linguagem / relógio	Python 3.13.5 / perf_counter_ns

Advertência de reprodutibilidade: os testes foram executados num contêiner Linux disponibilizado durante a geração do trabalho, não no computador pessoal da dupla. Limites de CPU, virtualização, frequência e ruído do ambiente podem alterar as constantes de tempo. Os CSVs, a configuração e as sementes estão registrados no repositório para reexecução local.

4. Resultados — subarranjo de soma máxima

n	Força bruta (ms)	Divisão (ms)	Aceleração
100	0,833	0,113	7,4×
200	3,496	0,212	16,5×
400	13,680	0,437	31,3×
800	58,360	0,984	59,3×
1.600	247,266	2,042	121,1×
3.200	1.017,482	4,332	234,9×
6.400	3.646,365	8,904	409,5×

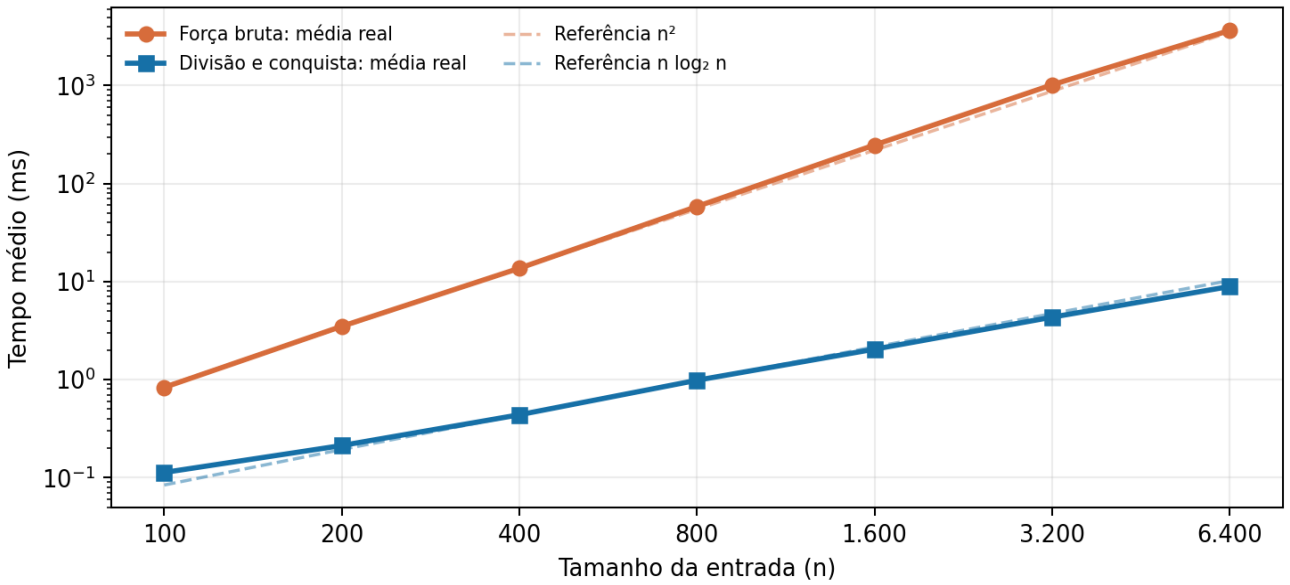


Figura 1. Médias de cinco execuções por tamanho; escala logarítmica em ambos os eixos. As linhas tracejadas são referências teóricas reescaladas no ponto n = 400, não medidas adicionais.

4.1. Interpretação

Para n = 100, a força bruta levou 0,833 ms, contra 0,113 ms da divisão e conquista (ganho de 7,4×). Em n = 6.400, os tempos passaram a 3.646,365 ms e 8,904 ms (ganho de 409,5×). O crescimento acelerado da curva quadrática e o menor declive da curva n log n são compatíveis com as recorrências; pequenas oscilações refletem custos de interpretação, alocação, cache e escalonamento.

5. Resultados — par de pontos mais próximo

n	Força bruta (ms)	Divisão (ms)	Aceleração
100	0,903	0,271	3,3×
200	3,086	0,683	4,5×
400	12,455	1,243	10,0×
800	60,289	3,426	17,6×
1.600	253,681	7,423	34,2×
3.200	929,534	13,864	67,0×
6.400	3.822,253	30,606	124,9×

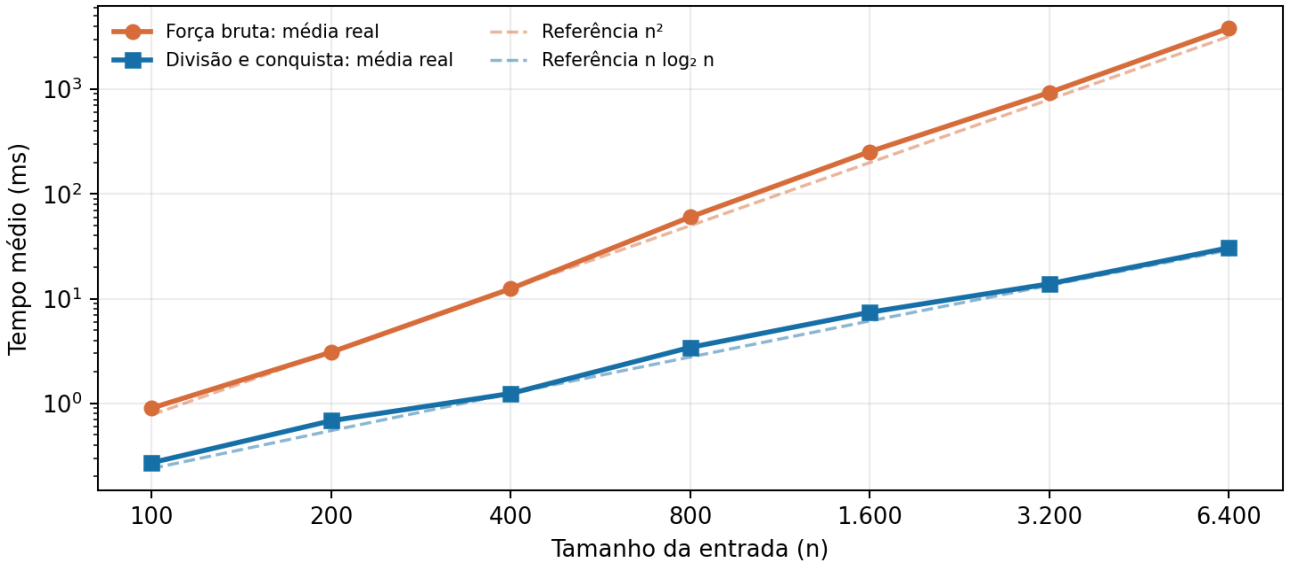


Figura 2. Comparação em escala log–log das medições reais com duas curvas de referência n² e n log₂ n, ajustadas em n = 400.

5.1. Seleção justificada do limiar

Limiar	n = 200	n = 800	n = 3.200
4	0,535	2,692	24,910
8 (adotado)	0,578	2,660	12,582
16	0,643	3,275	16,088
32	1,017	4,588	20,452
64	1,650	7,716	32,796

Tabela de calibração em ms (cinco repetições por célula). Critério: menor média geométrica das razões entre o tempo observado e o melhor tempo de cada tamanho, atribuindo peso igual aos três n. O limiar **8** apresentou a melhor pontuação nesse protocolo; a amostra não demonstra que ele seja ótimo em toda máquina.

6. Comparação, validade e conclusão

6.1. Crescimento e pontos de cruzamento

As duas soluções por força bruta realizam número quadrático de operações; as variantes recursivas mantêm o custo O(n log n) à custa de ordenação, particionamento ou pilha de chamadas. Para entradas pequenas, uma implementação exaustiva pode vencer por ter menos overhead e estruturas auxiliares, embora isso não tenha ocorrido nos sete tamanhos principais desta máquina. Os cruzamentos, quando existentes, dependem de compilador/interpretador, tamanho e distribuição da entrada, limiar, memória e carga de CPU; não são determinados apenas pela notação O.

6.2. Cobertura e correção

Verificação	Cobertura
Casos documentados	11 vetores e 11 coleções de pontos, com entrada e saída esperada em JSON.
Subarranjo	3.200 entradas pseudoaleatórias; saída completa comparada à força bruta.
Par de pontos	1.000 entradas pseudoaleatórias em seis limiares, mais 60 sem repetição.
Condições especiais	n mínimo, negativos, zeros, empates, cruzamento, x repetido, coincidência, colinearidade, valores altos e fracionários.
Entradas inválidas	Quantidade insuficiente, tipos impróprios e coordenadas não finitas.

6.3. Reprodutibilidade e limitações

O arquivo **resultados/medicoes_brutas.csv** contém os 140 tempos individuais (4 algoritmos × 7 tamanhos × 5 rodadas). O arquivo **resumo_medicoes.csv** contém médias e desvios; **calibracao_limiar.csv** registra as 75 medições de limiar; **ambiente_e_protocolo.json** detalha o hardware observável e as sementes. Os comandos completos constam do README.

A análise empírica verifica tendências no intervalo testado, não prova limites assintóticos sozinha. Tempos de uma máquina virtual contêm interferência e não podem ser apresentados como resultados de uma estação física da dupla. A implantação num repositório Git remoto e seu histórico distribuído precisam ser realizados pelos integrantes; as versões históricas de semanas anteriores não foram inventadas.

6.4. Conclusão

Ambos os problemas admitem soluções diretas $\Theta(n^2)$ e soluções de divisão e conquista $\Theta(n \log n)$. A teoria, a validação funcional e as medições são coerentes: conforme n cresce, a variante recursiva amplia a vantagem sobre força bruta. A Etapa 2 exige atenção adicional à ordenação por y, à faixa de até sete vizinhos, a pontos coincidentes e à escolha do limiar, abordados na implementação.

Referências didáticas: Enunciado “Trabalho Prático: Divisão e Conquista”; material da disciplina “Aula 05 — Teorema Mestre” e “Aula 07 — Introdução à Construção de Algoritmos: Divisão e Conquista”.